Autonomous Lane Following Vehicle Design

Team 16

Haaris Tahir-Kheli, Yanning Xu, Colin Tang, Maheshwari Tanwar

EEC195A&B: Senior Design Project

Professor Lance Halsted

5th June, 2025

Table of Contents

Project Objective

The goal of our project was to design, build, and perfect an autonomous lane-following vehicle that could navigate indoor or outdoor tracks using only onboard sensors and logic. As part of the UC Davis EEC195A&B senior design project, our mission was to implement a reliable, robust, and high-speed control system that leverages visual sensing and precise actuation to follow lanes with high accuracy. The project challenged us to combine hardware and software engineering in a real-world robotics application.

To achieve this, we used the OpenMV Cam RT1062 as our central controller. This microcontroller enabled us to capture live video, process visual information using built-in computer vision libraries, and directly control our motors based on the detected lane geometry. The OpenMV platform supported us in rapidly iterating our line and lane detection logic, as well as implementing PID (Proportional–Integral–Derivative) control for steering.

A critical aspect of the hardware was the custom motor control PCB we designed in Altium Designer and fabricated via JLCPCB. This board housed all necessary power regulation and motor driver circuits required for high-speed, high-current control of the Traxxas Rustler's brushed DC motor and steering servo. The layout incorporated precise grounding strategies such as star ground topology to isolate analog and digital noise, ensuring stability in motor control.

As a team, we:

- Learned foundational vision algorithms such as blob detection and regression-based line detection
- Wrote modular MicroPython code to process real-time visual data into control commands

- Designed and soldered a 2-layer motor control PCB
- Tuned and tested a PID controller for smooth lane centering behavior
- Fabricated and iterated a custom camera stand using 3D design and real-world stress testing
- Conducted over 1000 real-world test runs, including extensive overnight sessions to debug and optimize performance

Our final design combines machine vision, real-time control, and hardware design into a unified system capable of autonomously navigating a lane-marked track. The project represents not just an exercise in embedded systems and robotics, but a hands-on deep dive into solving complex engineering problems with a mix of creativity, teamwork, and grit.

Hardware Documentation

Our hardware journey was rooted in one critical challenge: designing a robust, noise-isolated, high-current-capable motor control system that could safely and responsively steer and drive our autonomous car, without compromising signal integrity or control precision. Working within the guidelines of Lab 8, our team successfully created a full-fledged 2-layer PCB using Altium Designer, implementing a mixed-signal control architecture, clean power distribution, and a carefully executed star grounding strategy.

System Overview: Block Diagram and Control Structure

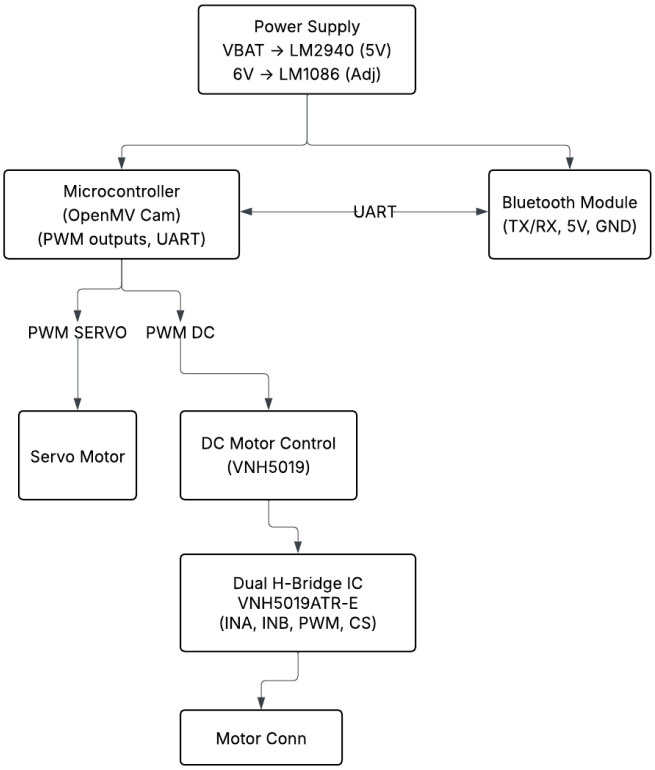

Figure 1: Block Diagram Showing Electrical Architecture

At a system level, our motor control board acts as the electrical backbone of the car. It powers and interfaces the following key subsystems:

- OpenMV Cam RT1062 (main controller)
- DC drive motor (for movement)
- Servo motor (for steering)
- HC-06 Bluetooth module (for wireless debugging and start/stop control)

Control signals from the OpenMV are routed to two alternate motor control paths — a high-performance H-bridge driver and a low-side MOSFET — while regulated voltage rails feed the camera, servo, and communication modules.

Circuit Schematic: Component Interfaces with OpenMV

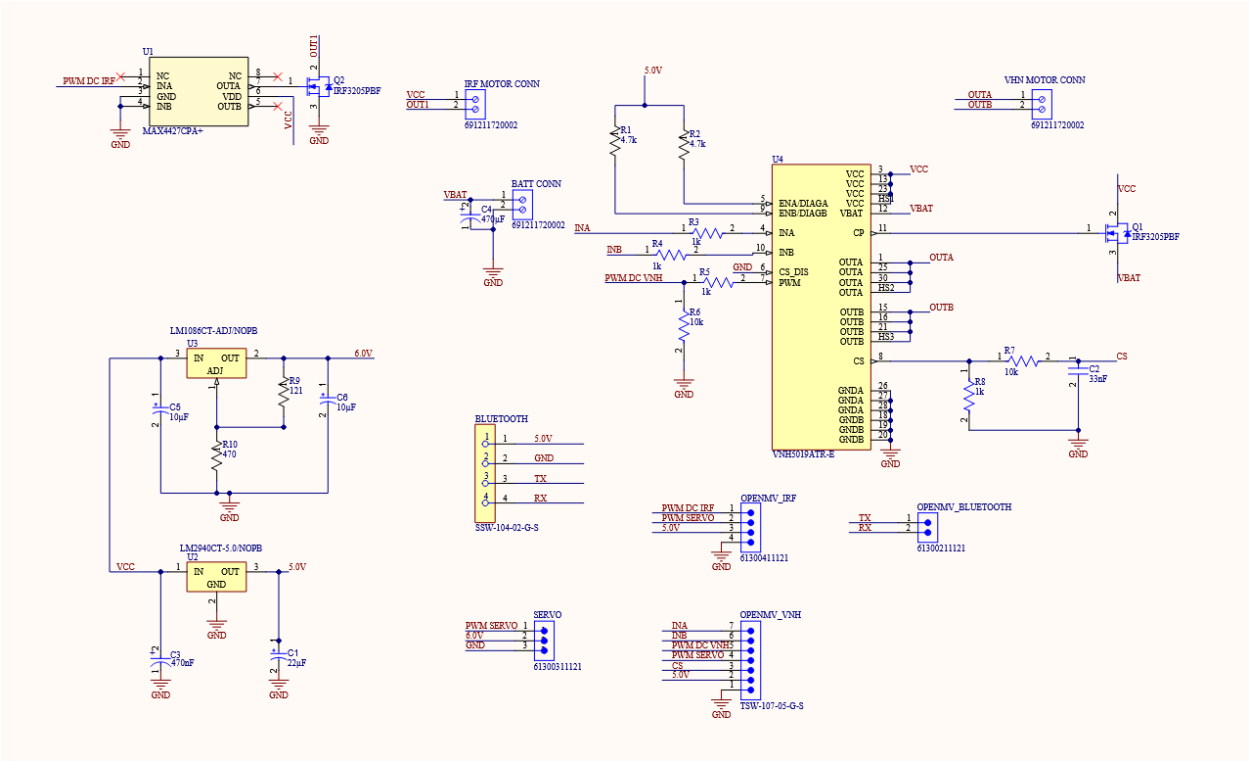

Figure 2: PCB Schematic Made on Altium Designer

The schematic includes clean routing of PWM signals and regulated power from the PCB to the OpenMV. This interface includes:

- PWM lines for motor speed (both H-Bridge and MOSFET)
- PWM line for servo control
- Current sense feedback (CS) from the H-Bridge
- 5V and GND rails provided via onboard LDO

This centralizes all key interactions on a single connector header, minimizing loose jumper wires and improving mechanical stability.

Motor Control Circuits: H-Bridge and MOSFET Redundancy

In alignment with the project requirement to support both control strategies for a brushed DC motor (up to 10A), our PCB includes two fully isolated motor control paths:

a) H-Bridge Control via VNH5019ATR-E

We implemented a complete control and current sense system using the VNH5019ATR-E H-Bridge IC. This high-side and low-side driver combo allows bidirectional motor control, receiving INA, INB, PWM, and CS signals via protective resistors. The CS output is filtered with a 10kΩ–33nF RC filter (R8–C2), providing an analog voltage representing motor current, which we feed back into the OpenMV for current monitoring.

The motor is connected via a dedicated 5.0mm terminal block, and power is routed through thickened traces directly to the VOUT and VBAT pins, as per the current demands.

b) Low-Side MOSFET Control using IRF3205 + MAX4427CPA+

As an alternative, we designed a low-side switching circuit using the IRF3205PBF N-channel MOSFET, with gate drive provided by the MAX4427CPA+ driver. This enables efficient PWM control with fast switching and thermal reliability.

The MOSFET's gate driver boosts a 3.3V PWM signal from the OpenMV to the full VBAT level, ensuring hard switching. A separate motor terminal block connects this circuit to the motor, and decoupling capacitors ensure switching stability.

As required, only one control path is active at a time. The inactive driver is held in its off state via logic low signals, ensuring no back-feeding or interference.

Power Regulation: Clean Voltage Rails via LDOs

Our design includes two LDO voltage regulators for clean, stable power delivery to sensitive subsystems:

- LM2940CT-5.0 provides a fixed 5V rail used by the OpenMV and HC-06 Bluetooth module. Capacitors (C3 = 0.47μF, C4 = 22μF) stabilize the output and support transient demands.
- LM1086CT-ADJ, configured with R9 = 470Ω and R10 = 121Ω, delivers a regulated 6.0V rail for the servo motor. Higher voltage improves servo torque and speed. C5 and C6 (10μF each) support this output under heavy load.

This two-regulator strategy avoids power rail coupling and keeps motor and logic domains electrically independent.

Star Grounding: Protecting Logic from Power Noise

One of the most critical aspects — and a direct requirement from Lab 8 — was implementing proper grounding to isolate noisy motor and servo currents from our microcontroller logic. We followed the star grounding principle from the course lectures and the “Staying Well Grounded” article by Hank Zumbahlen:

Three separate ground pours were created:

- GNDA: For motor/H-bridge currents

- GND_SERVO: For servo power return
- GND_LOGIC: For OpenMV, Bluetooth, and control logic

These pours are connected at a single star point, located near the battery connector. This reduces IR drops, crosstalk, and noise injection, and ensures signal integrity for PWM, CS, and UART lines.

As discussed in class, we treated GND not as “where we clip a probe” but as a physical, resistive, and inductive return path, managing it intentionally to prevent analog performance degradation.

Bluetooth and Servo Interfaces: We included a 4-pin HC-06 Bluetooth header with labeled pins (VCC, GND, RX, TX) to enable remote control. The servo motor is also connected via a clearly labeled 3-pin header, powered by the 6.0V rail and receiving a direct 3.3V PWM signal from the OpenMV (compatible with servo logic).

Motor Control PCB Layout and Design

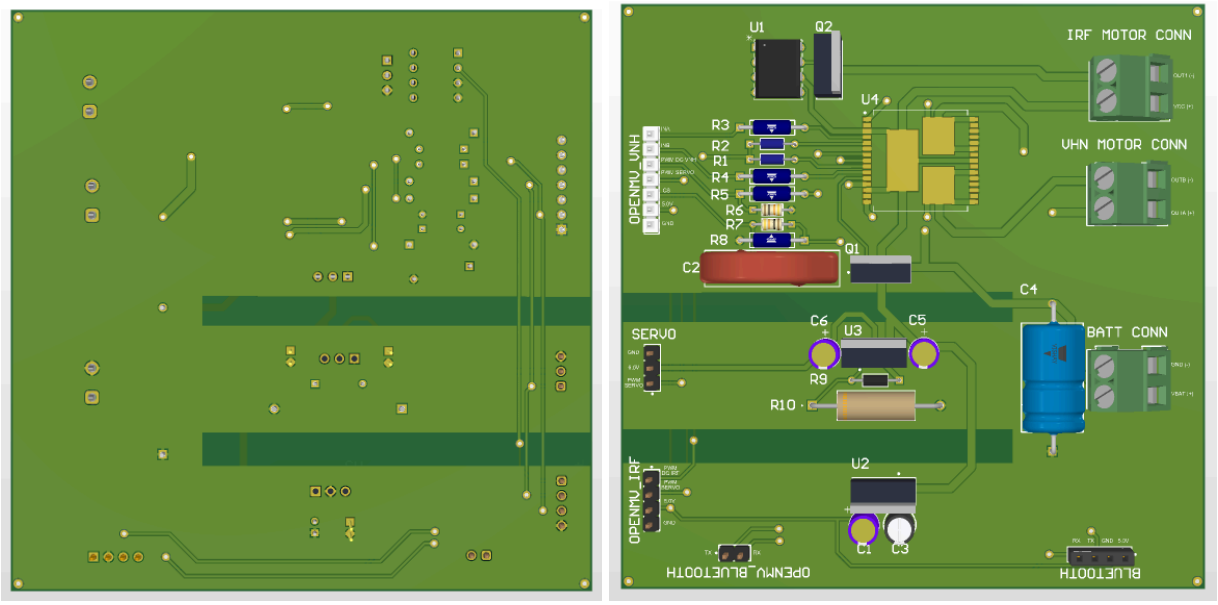

Figure 3: Front (a) and Back (b) of our PCB Layout On Altium Designer

Our 2-layer board was designed in Altium Designer with all constraints and DRC rules from Lab 8 fully implemented:

- Board dimensions under 100mm x 100mm
- All high-current traces (VBAT, motor outputs) sized for 10A using IPC-2221 guidelines
- Manual routing for all motor and power traces to minimize impedance and heating
- Silkscreen labels for all critical pins (VBAT, GND, INA, PWM, CS, etc.)
- Mounting holes in all four corners for mechanical stability
- OpenMV, HC-06, and servo headers placed on board edge for easy wiring

Our board passed all Altium DRC checks and was verified by TA before Gerber export.

Requirement	**Our Design Solution**
Battery interface via terminal block	VBAT connected using 5.0 mm terminal near the star ground region
H-Bridge and MOSFET motor control	Fully implemented with separate terminal blocks and independent signal paths
Regulated servo power (6V recommended)	LM1086CT-ADJ configured for 6.0V with proper capacitor decoupling
5V regulator for OpenMV & peripherals	LM2940CT-5.0 with decoupling capacitors and labeled output
Bluetooth interface	HC-06 header included with pin labels
OpenMV control interface	Headers for PWM, INA, INB, CS, 5V, and GND provided with clean routing

Star Grounding	Three isolated pours (Motor, Servo, Logic) joined at a single low-impedance point
Through-hole component preference	All resistors, capacitors, and regulators are through-hole, except VNH5019 (SMD)
Board ≤ 100x100 mm	Confirmed during layout and price quote from JLCPCB

Camera Mount Design and Iteration

Our camera mount went through several iterations as we balanced stability, adjustability, and real-world performance. Initially, we designed a simple, non-adjustable stand that fixed the camera at a predetermined angle. This version prioritized rigidity and performed well in reducing vibrations during motion. However, we quickly realized that having a fixed angle made it difficult to evaluate and optimize the camera's field of view for lane detection. Each time we wanted to test a different angle, we had to redesign and reprint the entire mount, which significantly slowed down our development process.

To overcome this limitation, we moved to a second design: an adjustable camera stand. This version allowed us to easily modify the camera's pitch and height during testing, which made it much easier to experiment with different configurations and identify the best angle for robust lane detection. The modular design proved invaluable for rapid iteration.

However, while the adjustability solved one problem, it introduced another — slight but noticeable jiggle during vehicle motion, which caused minor instability in the video feed. Initially, we suspected that the vibration stemmed from the base of the stand, thinking the flexible joints or printed parts were not sufficiently rigid.

After further investigation and multiple rounds of testing, we identified the true cause of the problem: the screws securing the mount to the car chassis. The original screws provided by the car were too loose and could not hold the stand firmly in place, leading to unintended movement during operation. Once we replaced them with standard M3 machine screws, the mount was securely fastened, and the vibration issues were eliminated.

This experience taught us the importance of mechanical integrity at every interface, not just in the mount design itself but also in how it connects to the broader system. The final version of the adjustable mount — printed in PETG for a balance of flexibility and strength — featured reinforced joints, truss cutouts for weight reduction, and a solid camera case to maintain alignment.

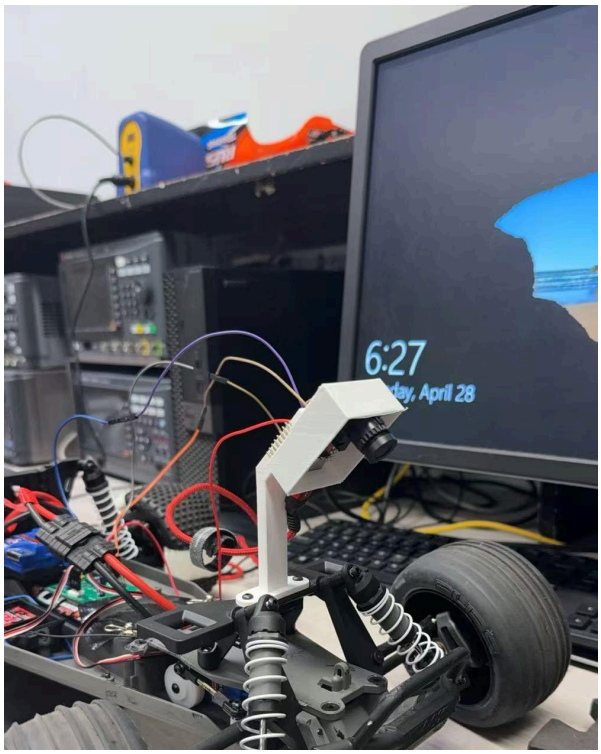

Figure 4: Initial Stand Design

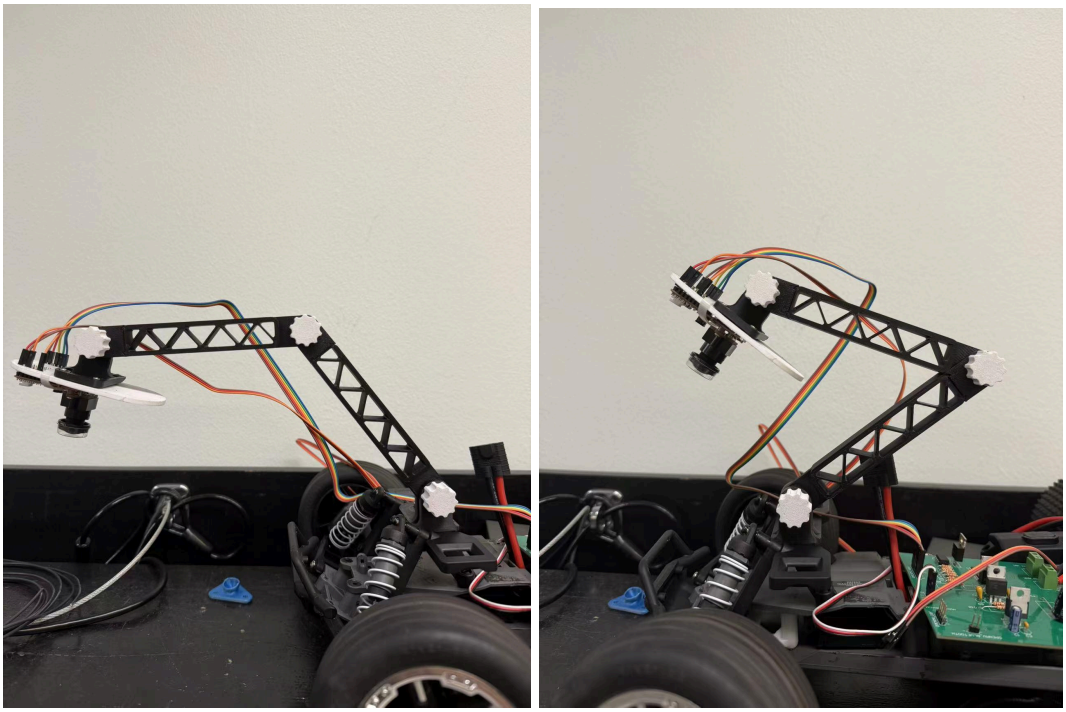

Figure 5: Different Angles We Tried For The Test

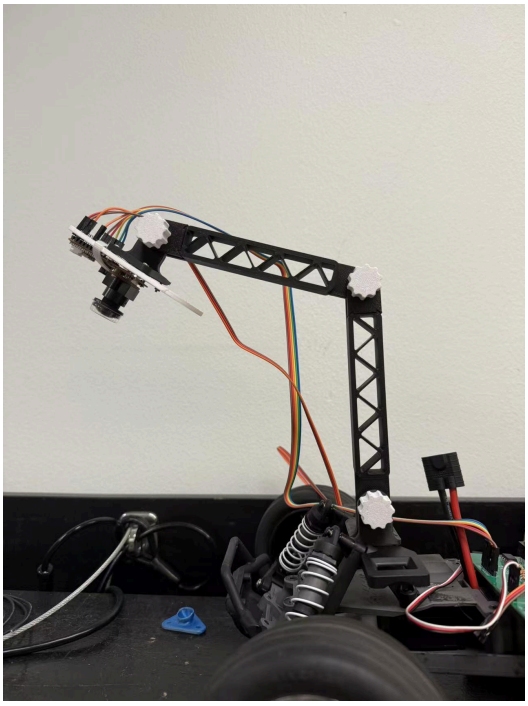

Figure 6: Final Angle Of The Stand

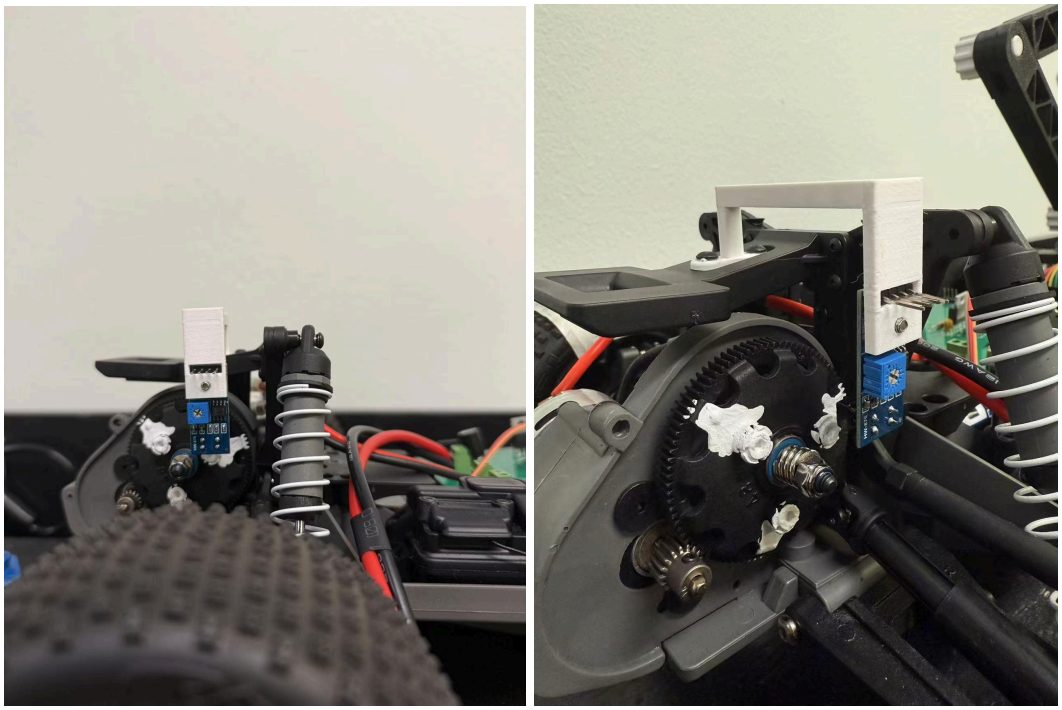

Figure 7: Final IR Sensor Design

Software Documentation

High-Level Description of the Lane-Following Code

(Only A Description, Pseudocode and Flowchart In The Sections Below)

The lane-following software functions as the control logic for our autonomous vehicle. It integrates real-time computer vision, closed-loop control through PID algorithms, and hardware-level pulse-width modulation (PWM) to ensure accurate steering and speed regulation. The implementation runs on an OpenMV microcontroller and interacts with peripheral hardware including a servo, brushed DC motor, IR sensor, and onboard LEDs.

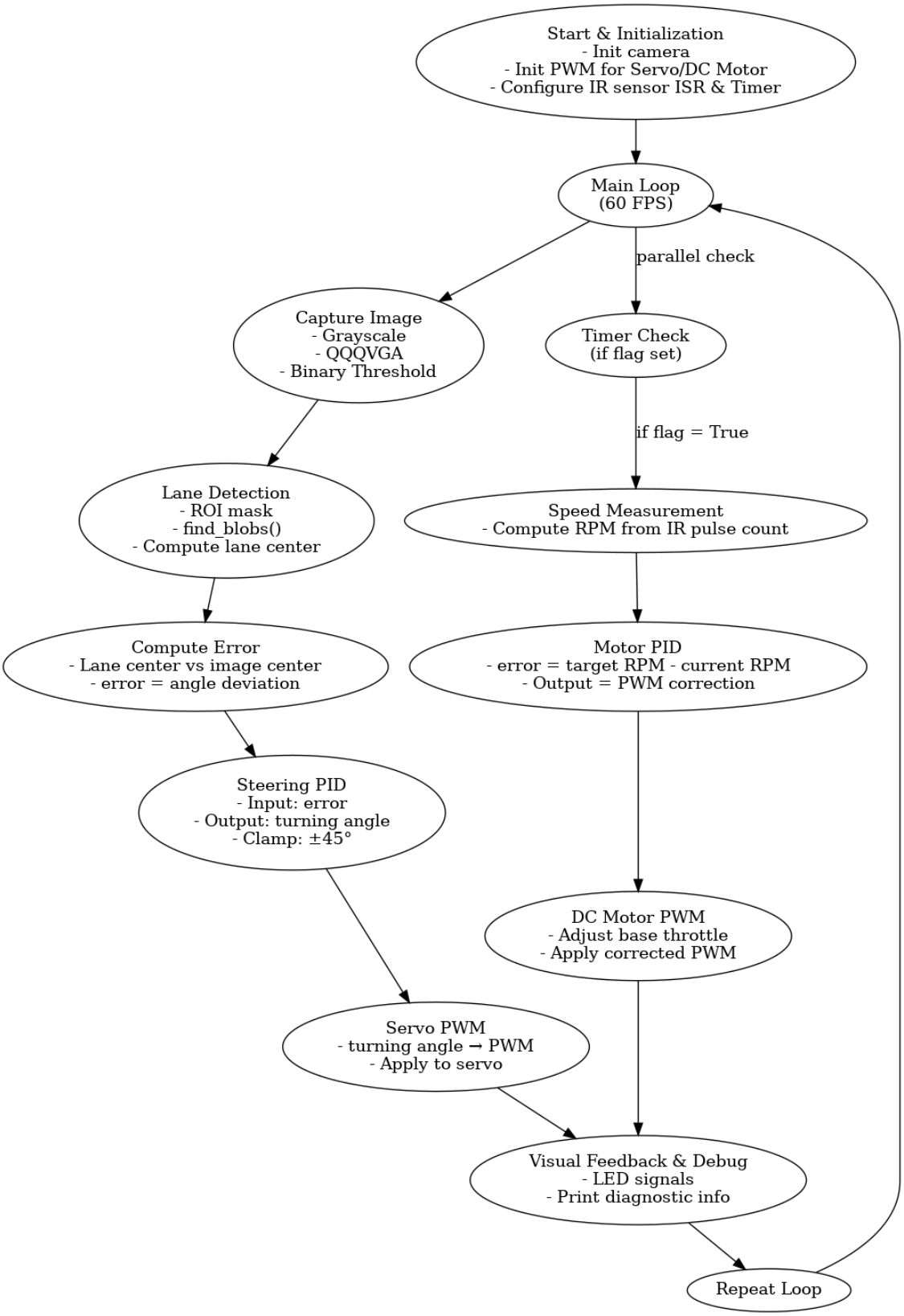

Figure 9: Programming Flow Diagram

Step 1: System Initialization and Hardware Setup

We began by importing all the necessary modules for our lane-following system. These included OpenMV's `sensor` and `time` libraries for camera control and timing, the `machine` module to interact with hardware components like PWM outputs, GPIO pins, and timers, and custom modules like `PID` and `helpers`. The `PID` module provides two independent controllers for steering and motor speed, while `helpers` contains utility functions for setting LED states and calculating servo pulse widths. The `const` module stores predefined configuration constants, and we also imported `isnan` to check for invalid float values during motor control.

At startup, all peripheral components are initialized. Since our goal was to detect white lane markings on a dark surface, color was unnecessary — we just needed contrast. Grayscale reduced the image to brightness values from 0 to 255, which simplified processing. We set the resolution to `QQQVGA` (80×60 pixels) to ensure fast frame capture and efficient computation. To maintain consistent brightness across frames, we disabled both auto gain and auto white balance, which are often unstable in dynamic lighting.

```
sensor.set_pixformat(sensor.GRAYSCALE)

sensor.set_framesize(sensor.QQQVGA)

sensor.set_auto_gain(False)

sensor.set_auto_whitebal(False)

sensor.skip_frames(time=2000)
```

Next, we configured PWM channels for both the steering servo and the motor. The motor direction was hardcoded to forward using GPIO pins `P10` and `P9`. Initial PWM duty cycles for the servo and motor were calculated using constants and scaled to match the expected duty resolution.

```
servo = PWM("P2", freq=const.SERVO_FREQ,
duty_u16=servo_duty_cycle_factor)

motor = PWM("P8", freq=const.MOTOR_FREQ,
duty_u16=motor_duty_cycle_factor)
```

To measure the speed of the car, we used an IR sensor mounted next to a slotted gear. Every time a slot passed the sensor, it triggered a falling-edge interrupt, incrementing a counter. Simultaneously, a periodic hardware timer set a flag at regular intervals. When this flag is set in the main loop, we calculate gear RPM based on tick count and time, and then derive wheel RPM using a gear ratio.

```
pin.irq(handler=isr, trigger=Pin.IRQ_FALLING)

tim = Timer(-1, mode=Timer.PERIODIC, period=const.TIMER_PERIOD,
callback=tick)
```

We also initialized three onboard LEDs (red, green, and blue) to provide visual feedback based on lane detection status. The `set_led()` helper function handles switching LEDs based on detection success or fallback mode. These initialization steps ensure all control signals, sensing components, and feedback mechanisms are ready before the real-time loop begins.

Step 2: Real-Time Lane Detection and Error Computation

Once initialized, the system enters a continuous control loop. In each iteration, the camera captures a grayscale frame, which is then converted into a binary image using a fixed threshold. This highlights lane lines as white blobs on a black background.

```
img = sensor.snapshot().binary([const.THRESHOLD])
```

To reduce processing load and focus on the road ahead, we define a **Region of Interest (ROI)** at the bottom 20 pixels of the frame. This region contains the part of the track the car is about to drive on.

```
roi = (0, const.FRAME_HEIGHT - 20, const.FRAME_WIDTH, 20)
```

```
blobs = img.find_blobs([(255, 255)], roi=roi, pixels_threshold=30, area_threshold=30, merge=True)
```

Using `find_blobs()`, we identify white pixel clusters (blobs) in the ROI. Each blob's center coordinates `(cx, cy)` are used to classify it as either a left or right lane marker, depending on its horizontal position relative to the frame center. For each side, we select the blob closest to the center — this reduces noise from irrelevant or faint blobs.

If both left and right blobs are found, we compute the **lane center** as the midpoint between their x-positions. The difference between this lane center and the frame center gives us the **steering error**, which represents how far the car has drifted from the ideal path.

```
error_steering = calc_error_lane(lane_center)
```

This error value is passed into the steering PID controller in the next step.

Step 3: Closed-Loop Steering and Speed Control via PID

We initialized two separate PID controllers — one for **steering** and another for **motor speed** — at the start of the program. The **steering PID** receives the error between the lane center and frame center and outputs a correction angle. This angle is clamped to ±45° to prevent extreme turning and is then mapped to a servo PWM value using a helper function.

```
turning_angle = min(max(p_i_d_steering.get_pid(error_steering), -45), 45)

steering_pw = calc_servo_pw(turning_angle)

servo.duty_ns(steering_pw)
```

If the lane is only partially detected (e.g., one blob missing), we reset the steering to the center and turn off the LEDs to enter a safe fallback mode.

In parallel, we perform motor speed control. If the timer flag is set, we calculate the current RPM using the IR sensor tick count, reset the count and flag, and compute the speed error:

```
error_motor = target_rpm - current_rpm

pw_adjust = min(max(p_i_d_motor.get_pid(error_motor), -5000), 5000)
```

This PID output is then added to the base motor PWM to create the final control signal. We also ensure it is valid by checking for `None` or `NaN`.

```
if pw_adjust is None or isnan(pw_adjust):

    pw_adjust = 0
```

```
motor_pw = base_motor_pw + pw_adjust

motor.duty_ns(int(motor_pw))
```

This approach creates a closed-loop system where both heading and speed are adjusted continuously based on visual and sensor feedback.

Step 4: Feedback, Indication, and Debugging

The system provides real-time visual feedback using onboard LEDs. If both left and right blobs are successfully detected, the **green LED is turned on** to indicate successful tracking. If either is missing, all LEDs are turned off to show the system is in fallback mode.

```
set_led("green", led_list)
```

We also output important runtime variables to the serial console for debugging and performance tuning. These include the lane center position, steering angle, motor power, and FPS, which help us monitor behavior during testing and tune PID parameters as needed.

```
print(f"Lane Center: {lane_center}, Turning: {turning_angle:.2f},
Motor PWM: {motor_pw}")
```

Pseudo-code of Main Software Modules

1. Initialization

```
Initialize camera in grayscale mode
```

```
Set frame size to QQQVGA

Disable auto gain and white balance

Skip 2000 ms for sensor to stabilize

Set motor pins: IN_A = HIGH, IN_B = LOW  // forward direction

Set up PWM for:

    - Motor on pin P1 with 100 Hz

    - Servo on pin P2 with 50 Hz

Set up IR pin interrupt on falling edge

Start timer to trigger RPM update every 100 ms

Initialize PID Controllers:

    - Steering_PID(kp_steer, ki_steer, kd_steer)

    - Motor_PID(kp_motor, ki_motor, kd_motor)

Wait for 5 seconds before main loop starts
```

2. Lane Detection & Lane Center Calculation

```
img ← capture image from camera
```

```
img ← convert to binary using threshold (e.g.,
img.binary([threshold]))
region ← define ROI at bottom of the frame
blobs ← find white blobs in region
left_blob  ← None
right_blob ← None
for each blob in blobs:
    if blob.cx() < CENTER_POSITION:
        left_blob ← blob  if it's closer to center than previous
    else if blob.cx() > CENTER_POSITION:
        right_blob ← blob  if it's closer to center than previous


if left_blob and right_blob exist:
    lane_center_x ← (left_blob.cx() + right_blob.cx()) / 2
    lane_center_y ← (left_blob.cy() + right_blob.cy()) / 2
```

3. Steering Error Calculation

```
x_offset ← CENTER_POSITION - lane_center_x

y_offset ← FRAME_HEIGHT - lane_center_y

error_angle ← atan2(x_offset, y_offset) * (180 / π)

if error_angle > 45: error_angle = 45

if error_angle < -45: error_angle = -45
```

4. Steering PID Controller (angle to servo PWM)

```
// PID Calculation (Steering)

P ← kp * error

I ← I + ki * trapezoidal_integral(error, dt)

D ← kd * (error_now - error_past) / dt

steering_angle ← P + I + D

Clamp steering_angle between -45° and +45°

servo_pwm ← map_angle_to_pwm(steering_angle)

// Using formula: pwm = STEERING_CENTER - ((STEERING_RIGHT -

STEERING_LEFT) * (steering_angle / 90))

Set servo duty cycle to servo_pwm
```

5. Motor RPM Measurement

```
If tick_flag == True:

    rotations ← pulse_count / PULSES_PER_ROTATION

    gear_rotations ← rotations / GEAR_RATIO

    RPM ← (gear_rotations / TIME_WINDOW_SEC) * 60

    Reset pulse_count to 0

    Clear tick_flag
```

6. Motor PID Controller (speed control)

```
error_speed ← TARGET_RPM - current_RPM

// PID Calculation (Motor)

P ← kp * error_speed

I ← I + ki * trapezoidal_integral(error_speed, dt)

D ← kd * (error_now - error_past) / dt

power_adjustment ← P + I + D

Clamp power_adjustment to safe range: [-max_delta, +max_delta]

motor_pwm ← BASE_MOTOR_PWM + power_adjustment
```

```
Set motor duty cycle to motor_pwm
```

7. LED Feedback

```
if left_blob and right_blob:
    set_led("green")
else if left_blob or right_blob:
    set_led("blue")
else:
    set_led("red")
```

8. Main Loop

```
while True:
    capture frame
    detect lane
    compute lane_center
    calculate steering error
    run steering PID and update servo
    if tick_flag is set:
```

```
        compute RPM

        run motor PID and update motor

    update LEDs

    print debug info
```

Design and Performance Summary

Our final autonomous car was able to successfully complete two full laps of the competition track in 49.3 seconds, placing us 7th overall. What truly stood out was our car's ability to maintain lane detection and steering precision through **both sharp hairpins and soft bends**, even as track lighting changed throughout the test runs.

Thanks to our tuned PID controllers, real-time feedback, and vibration-stable camera mount, we achieved **smooth path tracking** with minimal deviation.

Overall, our design was robust, fast, and responsive, showing that both our electrical and mechanical systems worked in sync — and delivered consistent performance when it mattered.

References / Acknowledgement of Sources

Special Thanks: Professor Lance Halsted, TAs: Mr. Damia Casas, Mr. Ning Miao, Mr. Hualong Yu for their support and lifelong teachings.

• OpenMV Documentation – https://docs.openmv.io

• Altium Designer Resources – https://resources.altium.com

• SolidWorks Tutorials – https://www.solidworks.com

• PID Control Concepts – https://controlguru.com

Lane Detection: Brief History and the Road Ahead | by Wajahat Kazmi | motive-eng | Medium